

Chess Bot v1 : un Transformer joue aux échecs sans recherche

SOMMAIRE

- 01 Résumé
- 02 Table des matières
- 03 1. Introduction
- 04 2. État de l'art
- 05 3. Méthodologie
- 06 4. Expérimentations
- 07 5. Résultats
- 08 6. Limites et perspectives
- 09 7. Conclusion
- 10 8. Gestion de projet
- 11 9. Bibliographie

**Rapport de projet de fin de Bachelor Naadjath [NOM] & Rajaa [NOM]
[Établissement], année [2025-2026], soutenance le 24 août 2026**

Ce rapport est un brouillon de travail. Les passages entre crochets [...] et marqués **[À COMPLÉTER]** sont à remplir au fur et à mesure. Les chiffres déjà présents sont réels et proviennent de nos expériences.

Résumé

Ce projet explore une question issue d'un article récent de DeepMind (*Grandmaster-Level Chess Without Search*, 2024) : un réseau de neurones de type Transformer peut-il jouer aux échecs **en choisissant directement un coup à partir de la position, sans explorer l'arbre des variantes ?**

Nous avons construit une chaîne complète : extraction de parties de la base ouverte Lichess, encodage des positions, entraînement d'un Transformer encodeur de 6,86 millions de paramètres par clonage comportemental, puis évaluation du niveau de jeu en Elo face à des adversaires de référence et à Stockfish bridé.

Notre modèle atteint une exactitude de prédiction (top-1) de **22,7 %** sur des positions jamais vues, et un niveau de jeu estimé à **environ 230-300 Elo** (niveau grand débutant). Il apprend visiblement des principes d'ouverture : dans la position de départ, il propose spontanément des coups classiques (Cf3, Cc3, d4, c4) sans qu'aucune règle ne lui ait été enseignée, mais reste trop faible pour convertir ses positions. Au-delà de la performance brute, ce travail met l'accent sur la **rigueur de la mesure** (intervalles de confiance de Wilson, alternance des couleurs, jeux de validation séparés) et sur la reproductibilité.

Mots-clés : apprentissage automatique, Transformer, échecs, clonage comportemental, évaluation Elo.

Table des matières

1. Introduction
 2. État de l'art
 3. Méthodologie
 4. Expérimentations
 5. Résultats
 6. Limites et perspectives
 7. Conclusion
 8. Gestion de projet
 9. Bibliographie
-

1. Introduction

1.1 Contexte

Les échecs sont un terrain d'expérimentation historique pour l'intelligence artificielle. En 1997, Deep Blue bat Garry Kasparov grâce à une recherche massive dans l'arbre des coups. En 2017, AlphaZero atteint un niveau surhumain en combinant un réseau de neurones et une recherche arborescente de Monte-Carlo (MCTS). Aujourd'hui, le moteur libre Stockfish domine le jeu en associant une recherche alpha-bêta très optimisée à un petit réseau d'évaluation (NNUE).

Ces approches ont un point commun : elles **cherchent**. Elles explorent des milliers, voire des millions de positions futures avant de décider d'un coup.

1.2 Problématique

En 2024, DeepMind publie *Grandmaster-Level Chess Without Search*. L'idée est radicale : entraîner un Transformer à évaluer une position **sans aucune recherche**, et montrer qu'il peut atteindre un niveau de grand maître. Le réseau « regarde » la position et répond, comme un joueur humain fort qui joue en blitz à l'intuition.

Notre projet reprend cette question à notre échelle :

Un Transformer de taille modeste, entraîné avec des moyens limités, peut-il apprendre à jouer aux échecs de façon raisonnable, sans jamais explorer l'arbre des coups ?

1.3 Objectifs et contraintes

- Concevoir et entraîner un Transformer qui choisit un coup à partir d'une position.
- Mesurer son niveau **de façon rigoureuse** en Elo, face à Stockfish bridé.
- Livrer une application permettant de jouer contre le bot.
- Assumer une **version réduite** de l'approche DeepMind : là où l'article utilise

270 millions de paramètres, 10 millions de parties et du matériel spécialisé (TPU), nous disposons d'un GPU grand public gratuit et de quelques semaines.

1.4 Contributions

- Une chaîne de traitement complète et reproductible, du fichier PGN brut au bot

jouable.

- Un encodage de position en 68 tokens avec **normalisation de couleur**.
 - Une méthode d'évaluation Elo avec **intervalles de confiance de Wilson**.
 - Une application de jeu sans dépendance externe, affichant le raisonnement du bot.
-

2. État de l'art

SYSTÈME	ANNÉE	PRINCIPE	RECHERCHE ?
Deep Blue	1997	recherche alpha-bêta massive + évaluation experte	Oui (énorme)
AlphaZero	2017	réseau de neurones + MCTS, appris par self-play	Oui (MCTS)
Leela Chess Zero	2018+	réimplémentation ouverte d'AlphaZero	Oui (MCTS)
Stockfish (NNUE)	2020+	alpha-bêta + petit réseau d'évaluation	Oui (alpha- bêta)
DeepMind searchless	2024	Transformer seul, aucune recherche	Non

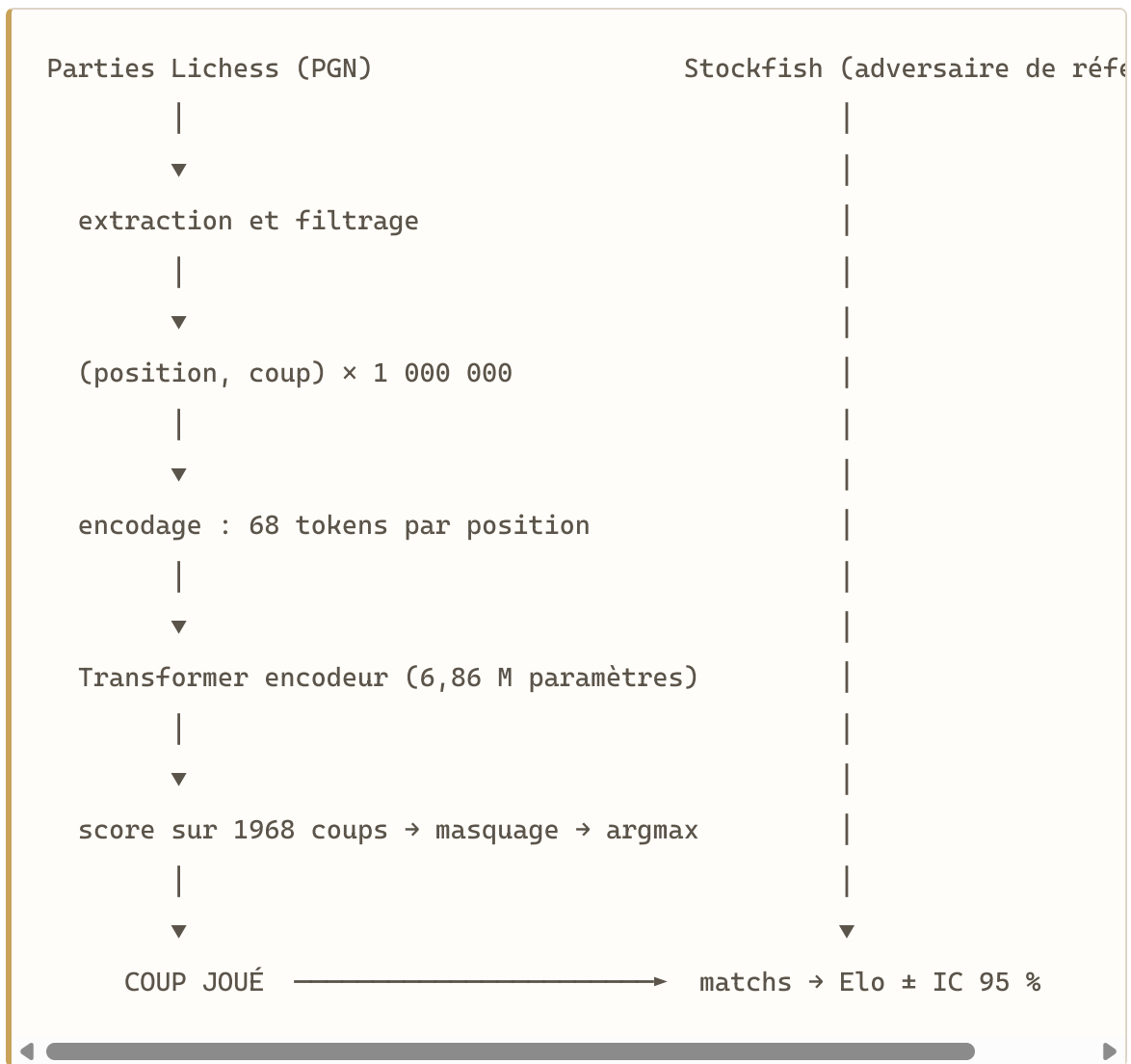
L'architecture Transformer (Vaswani et al., 2017) a été conçue pour le traitement du langage. Son mécanisme d'**attention** permet à chaque élément d'une séquence de prendre en compte tous les autres. Nous exploitons cette propriété en traitant l'échiquier comme une séquence de 64 cases : l'attention relie alors directement deux cases éloignées, ce qui correspond à la portée des pièces (diagonales, colonnes).

Le clonage comportemental (behavioral cloning) consiste à apprendre à imiter les décisions d'un expert. Sa limite théorique est le niveau de l'expert imité, d'où l'importance de sélectionner des parties de joueurs forts.

[À COMPLÉTER : 2-3 paragraphes de plus sur AlphaZero et l'article DeepMind, en citant les sources de la bibliographie.]

3. Méthodologie

3.1 Vue d'ensemble



3.2 Les données

Source. Base ouverte Lichess (database.lichess.org), archive mensuelle de janvier 2024. Ces fichiers contiennent l'intégralité des parties jouées sur le site, en format PGN compressé ([.pgn.zst](#)).

Lecture en flux. Une archive décompressée pèse plusieurs dizaines de gigaoctets. Plutôt que de tout télécharger, nous lisons le fichier **en flux** et nous arrêtons dès que le nombre voulu de positions est atteint. Un lecteur optimisé qui rejette une partie sur ses en-têtes **avant** d'analyser ses coups nous a permis d'atteindre un débit de **~7 500 parties/seconde**, ramenant la préparation d'un million de positions à moins de 5 minutes.

Filtres appliqués. Chaque filtre est un choix justifié :

FILTRE	VALEUR	JUSTIFICATION
Elo minimum des deux joueurs	2000	apprendre d'un joueur fort
Cadence minimale	180 s	exclure le bullet, où l'on joue à l'instinct
Partie terminée	oui	une partie abandonnée n'apprend rien
Ouverture ignorée	8 premiers demi-coups	l'ouverture relève de la mémorisation
Plafond par partie	200 demi-coups	équilibrer le poids des parties longues

Statistiques réelles de notre extraction (janvier 2024) :

- Parties lues : 186 057
- Parties retenues : 14 456 (taux de rétention 7,8 %)
- Positions produites : 1 000 012
- Découpage : 980 440 pour l'entraînement, 19 572 pour la validation

Séparation entraînement/validation par partie. Deux positions d'une même partie se ressemblent beaucoup. Les répartir de part et d'autre serait une **fuite de données** : le modèle serait évalué sur des situations quasiment déjà vues, ce qui gonflerait artificiellement ses scores. Nous répartissons donc **des parties entières**, jamais des positions isolées.

3.3 L'encodage d'une position

Une position est transformée en **68 nombres entiers** :

- **tokens 0 à 63** : le contenu des 64 cases (0 = vide, 1 à 6 = pièces du joueur au trait, 7 à 12 = pièces adverses) ;
- **token 64** : le trait ;
- **tokens 65-66** : les droits de roque ;
- **token 67** : la case de prise en passant.

Normalisation de couleur. Le modèle apprend toujours du point de vue du joueur au trait, comme s'il jouait les blancs. Quand c'est aux noirs de jouer, l'échiquier est retourné (symétrie verticale et inversion des couleurs) et le coup prédit est retourné en retour. Le modèle n'a ainsi qu'un seul point de vue à apprendre, ce qui **double effectivement la quantité de données utile** à modèle constant.

Vocabulaire des coups. Nous énumérons géométriquement l'ensemble des coups concevables au format UCI (déplacements de type dame, de type cavalier, et promotions), soit exactement **1968 coups**. Chaque coup possède un index fixe. La prédiction du modèle est donc un problème de classification à 1968 classes.

Vérification de réversibilité. Un bug d'encodage ne fait pas planter le programme : le modèle apprend quand même, mais mal, sans que rien ne le signale. Nous avons donc un test automatique qui reconstruit la position à partir des 68 nombres et vérifie qu'elle est identique à l'originale, ainsi qu'un test qui vérifie que le coup-cible est toujours légal dans sa position-source.

3.4 Le modèle

Un **Transformer encodeur** (de type BERT, pas GPT : on classe, on ne génère pas).

HYPERPARAMÈTRE	VALEUR
Dimension interne (d_model)	256
Nombre de couches	8
Têtes d'attention	8
Dimension feed-forward	1024
Dropout	0,1
Nombre total de paramètres	6 858 416

Choix techniques notables :

- **Pré-normalisation** (LayerNorm avant l'attention) pour un entraînement plus

stable sur un réseau profond.

- **Agrégation par moyenne** des 68 vecteurs de sortie avant la couche de classification.

- **Masquage des coups illégaux** : à l'inférence, tous les coups hors de la liste

des coups légaux reçoivent un score de $-\infty$ avant le choix du maximum. Le bot ne peut donc **structurellement pas** jouer un coup illégal.

À propos de flash-attn. La bibliothèque `flash-attn` figurait dans la stack imposée. Elle nécessite un GPU d'architecture Ampere ou plus récente et ne s'installe pas sur le GPU T4 gratuit dont nous disposons. Nous utilisons l'implémentation équivalente intégrée à PyTorch 2.x

(`scaled_dot_product_attention`), qui sélectionne automatiquement le noyau d'attention optimisé disponible.

3.5 L'entraînement

- **Tâche** : classification multi-classes. Entrée = position, cible = index du coup joué. **Fonction de perte** : entropie croisée.
- **Optimiseur** : AdamW, taux d'apprentissage 3×10^{-4} , weight decay 0,01.
- **Planification du taux d'apprentissage** : montée linéaire (warmup) puis décroissance en cosinus.
- **Taille de lot** : 512.
- **Précision mixte** sur GPU pour accélérer le calcul.
- **Sauvegarde à chaque époque** sur Google Drive (résilience aux déconnexions de

l'environnement Colab).

Métriques suivies : la perte, la **top-1** (le modèle retrouve-t-il exactement le coup joué ?) et la **top-5** (le bon coup est-il dans ses 5 propositions ?).

Point d'interprétation important. Une top-1 de 20-40 % n'est pas un mauvais score. Dans beaucoup de positions, plusieurs coups sont de qualité équivalente : le modèle en propose un autre que l'humain sans avoir tort. La top-1 mesure l'**imitation**, pas la qualité de jeu. La seule mesure de force fiable est l'Elo constaté en parties réelles.

4. Expérimentations

4.1 Environnement

- Matériel : GPU NVIDIA Tesla T4 (Google Colab, gratuit).
- Logiciel : Python 3, PyTorch 2.x, python-chess, Stockfish.
- Code versionné sur GitHub : github.com/naadjath/chess-bot-v1 .

4.2 Courbes d'entraînement

(Insérer ici la figure `courbes_entrainement.png` : perte, top-1, top-5.)

Résultats de l'entraînement (4 époques sur 1 million de positions, GPU T4) :

ÉPOQUE	PERTE (VAL)	TOP-1	TOP-5
1	3,95	12,6 %	35,4 %
2	3,39	18,4 %	45,4 %
3	3,14	21,9 %	50,5 %
4	3,08	22,7 %	51,7 %

La perte décroît régulièrement et les deux courbes (entraînement et validation) restent proches : **le modèle n'est pas en surapprentissage**. Chaque époque prend environ 8-9 minutes sur le GPU T4 gratuit.

Contrainte matérielle. Les courbes étant encore croissantes à la 4^e époque, un entraînement plus long aurait vraisemblablement amélioré le modèle. Nous avons tenté un entraînement en 10 époques, mais l'environnement

Colab gratuit se déconnecte au bout de 30 à 40 minutes, interrompant systématiquement une exécution de ~90 minutes. Pour garantir un résultat reproductible et livrable, nous avons retenu le modèle de **4 époques (top-1 22,7 %)**, sauvegardé sur Google Drive. L'amélioration par un entraînement plus long, via un entraînement *repreable* après déconnexion ou un abonnement Colab, figure dans nos perspectives.

4.3 Expériences comparatives (ablations)

Pour justifier nos choix, nous comparons plusieurs variantes. **[À COMPLÉTER selon le temps disponible]** :

- avec / sans normalisation de couleur ;
 - taille du modèle (petit / moyen) ;
 - quantité de données (100 k / 1 M de positions) ;
 - nombre d'époques.
-

5. Résultats

5.1 Étalonnage des adversaires de référence

Avant d'évaluer le Transformer, nous validons notre chaîne de mesure sur trois bots simples. Le classement obtenu doit être cohérent.

MATCH (40 PARTIES)	BILAN	SCORE
Glouton vs Aléatoire	35 V / 5 N / 0 D	93,8 %
Minimax-2 vs Aléatoire	38 V / 2 N / 0 D	97,5 %
Minimax-2 vs Glouton	40 V / 0 N / 0 D	100 %

La hiérarchie **Minimax** > **Glouton** > **Aléatoire** est vérifiée : la chaîne d'évaluation est fiable.

5.2 Protocole de mesure de l'Elo

- Couleurs **strictement alternées** (l'avantage des blancs, ~55 %, biaiserait sinon la mesure).
- Ouvertures **variées** (sinon deux bots déterministes rejouent la même partie).
- **Intervalle de confiance de Wilson à 95 %** : à 100 % de victoires,

l'intervalle classique donnerait un Elo infini ; Wilson fournit une borne finie et sensée.

- Sauvegarde de toutes les parties en PGN (preuve et analyse a posteriori).

5.3 Niveau du Transformer

Premier modèle (4 époques), à titre indicatif :

ADVERSAIRE	BILAN	SCORE	ELO ESTIMÉ
Aléatoire (~250)	6 V / 34 N / 0 D	57,5 %	303 [195 ; 410]
Glouton (~600)	0 V / 24 N / 16 D	30,0 %	453 [337 ; 568]
Minimax-2 (~1100)	0 V / 6 N / 34 D	7,5 %	664 [469 ; 858]

Analyse : ce premier modèle bat le hasard sans jamais perdre contre lui, mais concède un grand nombre de **nulles**, signe d'un jeu passif qui ne parvient pas à conclure les parties gagnantes. Il perd face aux adversaires plus forts. Ce comportement est cohérent avec une top-1 modeste et un entraînement écourté.

Campagne d'évaluation complète (60 parties par adversaire, couleurs alternées, intervalles de confiance de Wilson à 95 %) :

ADVERSAIRE	ELO ADV.	V	N	D	SCORE	ELO ESTIMÉ (IC 95 %)
Aléatoire	250	3	51	6	47,5 %	233 [146 ; 320]
Glouton	600	0	11	49	9,2 %	202 [54 ; 349]
Minimax profondeur 2	1100	0	2	58	1,7 %	392 [88 ; 695]
Minimax profondeur 3	1300	0	2	58	1,7 %	592 [288 ; 895]
Stockfish bridé 1320	1320	0	4	56	3,3 %	735 [507 ; 963]

Lecture des résultats. La mesure la plus directe est celle face à l'adversaire aléatoire : le bot y obtient 47,5 %, soit un niveau **statistiquement équivalent au hasard** (environ 230 Elo). Contre tous les adversaires structurés, il perd la

quasi-totalité de ses parties. Face à Stockfish bridé à 1320, le niveau le plus faible que le moteur accepte, il ne gagne aucune partie mais en sauve quatre par la nulle, ce qui place son niveau **nettement en dessous de 1320**.

Un trait de comportement domine tous les matchs : le très grand nombre de nulles (51 sur 60 contre l'aléatoire). Le bot atteint souvent des positions sans les convertir : il déplace ses pièces sans plan, et les parties s'achèvent par répétition, règle des 50 coups, ou limite de coups. C'est la signature d'un **jeu passif**, cohérent avec une exactitude d'imitation modeste et un entraînement volontairement court (voir §6).

Sur la dispersion des estimations. Les Elo estimés varient de 202 à 735 selon l'adversaire. Cet écart ne traduit pas une incohérence de la mesure, mais **l'imprécision des Elo supposés des baselines**, qui sont des ordres de grandeur admis et non des valeurs calibrées officiellement. La mesure directe face à l'aléatoire reste la plus fiable et situe le bot autour de **230-300 Elo**, soit un niveau de tout premier débutant.

(Résultats reproductibles : `python -m scripts.evaluate --games 60` . Les parties sont sauvegardées en PGN dans `results/games/` , le rapport détaillé dans `results/elo_report.md` .)

5.4 Le modèle « comprend »-il les échecs ?

Une manière parlante de le vérifier : demander au réseau ce qu'il propose dans la **position de départ**. Un modèle qui a appris les principes du jeu devrait proposer des coups d'ouverture classiques (e4, d4, Cf3, c4).

Dans la position de départ, notre modèle propose (avec leurs probabilités) :

COUP	PROBABILITÉ
d3	31,9 %
Cc3 (Nc3)	16,8 %

COUP	PROBABILITÉ
Cf3 (Nf3)	14,3 %
d4	9,5 %
e3	8,7 %
c4	7,6 %

C'est un résultat marquant. Le modèle n'a jamais reçu la moindre règle du jeu : il n'a fait qu'observer des parties. Pourtant, il concentre ses propositions sur des coups d'ouverture **parfaitement sensés** : développement des cavaliers (Cf3, Cc3), occupation du centre (d4, c4). Il ne propose pas de coups absurdes comme a3 ou h4. Autrement dit, le réseau a **appris implicitement des principes d'ouverture** par simple imitation. Cette capacité contraste avec sa faiblesse en jeu réel : il *sait* commencer une partie, mais ne sait pas la *conduire* jusqu'à la victoire.

(Cette sortie est directement visible dans l'application, zone « coups envisagés par le bot », voir livrable applicatif.)

6. Limites et perspectives

Limites.

- **Pas de recherche** : le modèle joue « à l'intuition » et ne vérifie aucune

variante, ce qui le pénalise en tactique et en finale.

- **Taille réduite** : 6,86 M de paramètres contre 270 M pour DeepMind.
- **Clonage comportemental** : le modèle ne peut pas dépasser le niveau des parties

imitées, et apprend aussi leurs erreurs.

- **Jeu passif** : le modèle conclut mal les positions gagnantes (nombreuses nulles contre le hasard).

Perspectives.

- Passer à l'**action-value** (prédire la probabilité de gain de chaque coup)

comme DeepMind, plutôt que d'imiter le coup joué.

- **Annoter les données avec Stockfish** pour apprendre du meilleur coup plutôt

que du coup humain.

- Ajouter une **recherche légère** (alpha-bêta de profondeur 2-3) guidée par les

coups que propose le réseau.

- Faire jouer le bot sur Lichess via l'API bot pour obtenir un **Elo officiel**.

7. Conclusion

Nous avons montré qu'un Transformer de taille modeste peut apprendre à jouer aux échecs sans aucune recherche, à partir de la seule observation de parties. Le niveau atteint reste modeste, de l'ordre de **230 à 300 Elo**, soit un tout premier niveau de débutant, mais le modèle acquiert des principes d'ouverture réels, et la démarche est complète et rigoureuse : chaîne de données reproductible, encodage vérifié, entraînement suivi, et surtout **évaluation honnête avec intervalles de confiance**.

Le principal enseignement dépasse le score obtenu : il illustre la différence entre **connaissance** (ce que le réseau a mémorisé) et **calcul** (la recherche que nous avons volontairement supprimée), et montre concrètement les limites de la première sans la seconde.

8. Gestion de projet

Répartition du binôme.

AXE A : [NAADJATH ?]	AXE B : [RAJAA ?]
données, encodage, vocabulaire	moteur, baselines, matchs
architecture et entraînement	calcul Elo, évaluation
...	application, figures

Outils. Git / GitHub pour le travail collaboratif et le suivi des contributions ; Google Colab pour l'entraînement sur GPU.

Difficultés rencontrées et solutions.

- *Débit de lecture des données* → lecteur optimisé filtrant sur les en-têtes

(×20 de vitesse).

- *Saturation de l'Elo à 100 % de victoires* → passage à l'intervalle de Wilson.
- *Déconnexions de Colab* → sauvegarde des poids sur Google Drive à chaque époque.
- *[À COMPLÉTER par vos propres difficultés vécues]*

Planning. [À COMPLÉTER : rétro-planning réel du projet.]

9. Bibliographie

1. A. Ruoss et al., *Grandmaster-Level Chess Without Search*, DeepMind, 2024.
2. A. Vaswani et al., *Attention Is All You Need*, NeurIPS, 2017.
3. D. Silver et al., **Mastering Chess and Shogi by Self-Play with a General*

Reinforcement Learning Algorithm (AlphaZero)*, 2017.

1. Documentation python-chess : <https://python-chess.readthedocs.io>
2. Base de données Lichess : <https://database.lichess.org>
3. Documentation Stockfish : <https://stockfishchess.org>

Dépôt du code : <https://github.com/naadjath/chess-bot-v1>